

DISCIPLINA · AULA 01

Integração e Entrega Contínua

CI/CD

Introdução à disciplina: conceitos, práticas e ferramentas que automatizam o caminho do código até a produção.



O que vamos ver hoje

01 O problema

Por que integrar software manualmente dói

02 Integração Contínua

O que é CI e como funciona na prática

03 Entrega e Implantação

As duas faces do CD

04 O pipeline

Etapas, testes e automação

05 Ferramentas

O ecossistema que você vai usar

06 A disciplina

Conteúdo, projetos e avaliação

Antes do CI/CD: o "inferno da integração"

Código isolado por semanas

Cada pessoa trabalha na sua branch e só junta tudo no fim.

Integração gigante e arriscada

Conflitos, bugs escondidos e madrugadas de correção.

Entrega manual e rara

Publicar exige checklist, planilha e alguém com "a senha".

Feedback tardio

O erro só aparece dias depois de ter sido escrito.



Integração Contínua (CI)

Prática em que a equipe integra o código ao repositório principal com frequência — idealmente várias vezes ao dia — e cada integração dispara um build e testes automáticos.

Commits pequenos e frequentes

Mudanças menores geram conflitos menores e são mais fáceis de revisar e reverter.

Build automatizado

Um único comando compila, empacota e valida o projeto — sem passos manuais.

Feedback em minutos

Se algo quebra, a equipe descobre imediatamente e corrige enquanto o contexto está fresco.

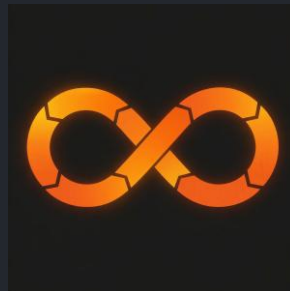
Entrega Contínua x Implantação Contínua

Entrega Contínua

Continuous Delivery — todo código aprovado fica sempre pronto para ir a produção. A publicação existe, mas é uma decisão humana: um clique.

Implantação Contínua

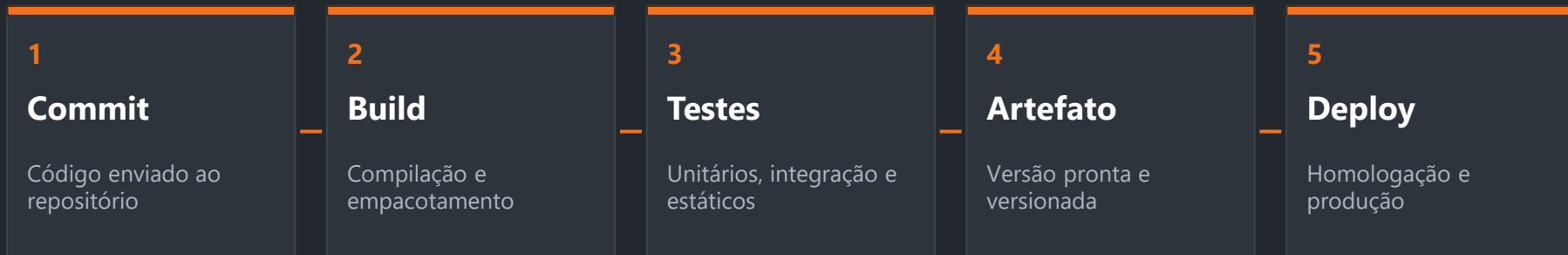
Continuous Deployment — não há clique: se o pipeline passa em todos os testes, a mudança vai automaticamente para produção.



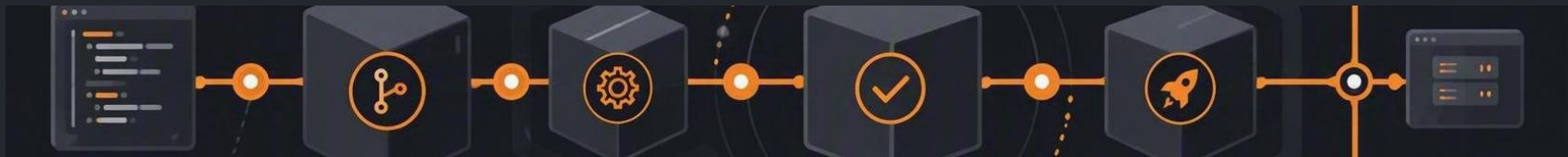
Em resumo

CI cuida de integrar e validar o código. CD cuida de levá-lo até o usuário com segurança e repetibilidade. As duas siglas juntas descrevem o caminho completo: do commit à produção.

O pipeline CI/CD passo a passo



Cada etapa só acontece se a anterior passou. Uma falha interrompe o pipeline e avisa a equipe imediatamente — nada quebrado avança para a próxima fase.



Práticas que sustentam o CI/CD

Controle de versão único

Um repositório é a fonte da verdade — código, scripts e configuração.

Build reproduzível

Qualquer pessoa (ou máquina) gera a mesma versão com um comando.

Testes automatizados

Rápidos, confiáveis e executados a cada mudança.

Branches de vida curta

Integre em horas ou dias, não em semanas.

Ambiente igual ao de produção

Containers e infraestrutura como código reduzem o "na minha máquina funciona".

Consertar o build é prioridade

Pipeline vermelho para a equipe: ninguém segue em frente.

Testes e portões de qualidade

Testes unitários

Verificam pequenas partes do código. São a base: muitos e muito rápidos.

Testes de integração

Checam se os módulos e serviços conversam corretamente.

Testes de ponta a ponta

Simulam o usuário real. Poucos, porque são lentos e frágeis.

Análise estática

Estilo, complexidade, vulnerabilidades e cobertura, sem executar o código.



O ecossistema de ferramentas

Versionamento

Git, GitHub, GitLab, Bitbucket

Servidores de CI/CD

GitHub Actions, GitLab CI, Jenkins, Azure DevOps

Build e dependências

Maven, Gradle, npm, Make

Testes e qualidade

JUnit, Jest, Pytest, SonarQube

Empacotamento

Docker, registries de imagem

Execução e infraestrutura

Kubernetes, Terraform, provedores de nuvem

Na disciplina usaremos Git + GitHub Actions + Docker como base — os conceitos valem para qualquer ferramenta.

Por que isso importa

Frequência de deploy

Entregas menores e mais constantes

Tempo até produção

Do commit ao usuário em horas, não meses

Taxa de falhas

Menos erros chegam ao ambiente real

Tempo de recuperação

Reverter ou corrigir em minutos

Essas quatro métricas (conhecidas como métricas DORA) são a forma mais usada para medir a maturidade de entrega de uma equipe.

- Menos retrabalho e menos conflitos de código
- Qualidade verificada de forma automática e constante
- Mais confiança da equipe para mudar o sistema
- Processo documentado no próprio pipeline

Desafios e armadilhas comuns

Pipeline lento

Se demora 40 minutos, ninguém espera. Paralelize e use cache.

Testes instáveis

Testes que falham sem motivo destroem a confiança no pipeline.

Automatizar o caos

CI/CD não conserta um processo ruim — ele o acelera.

Segredos expostos

Senhas e chaves nunca no repositório: use cofres de segredos.

Ambientes diferentes

O que passa em teste precisa se comportar igual em produção.

Cultura

É prática de equipe, não a tarefa de uma pessoa só.

Como será a disciplina

Conteúdo

- Fundamentos de Git e fluxo de trabalho
- Build e testes automatizados
- Pipelines como código
- Contêineres e ambientes
- Estratégias de deploy
- Monitoramento e feedback

Formato das aulas

- Teoria curta seguida de prática
- Laboratórios no seu repositório
- Discussão de casos reais
- Todo material no ambiente da turma

Avaliação

- Laboratórios práticos ao longo do curso
- Projeto final: uma aplicação com pipeline completo
- Participação e revisão entre colegas

PRÓXIMA AULA

Do commit ao primeiro pipeline

Traga seu ambiente pronto: Git instalado, uma conta no GitHub e um editor de código. Vamos criar juntos o pipeline mais simples possível — e vê-lo rodar.

Quiz: teste o que você aprendeu (1/2)

Cinco questões rápidas de múltipla escolha. Responda mentalmente e confira o gabarito comentado no final.

Questão 1

Qual é o objetivo principal da Integração Contínua?

- a) Publicar automaticamente em produção
- b) Integrar e validar o código com frequência
- c) Substituir os testes por documentação

Questão 2

Qual a diferença entre Entrega e Implantação Contínua?

- a) Não existe diferença
- b) Na Entrega, publicar é uma decisão humana
- c) A Implantação dispensa testes

Questão 3

O que acontece quando uma etapa do pipeline falha?

- a) O pipeline para e avisa a equipe
- b) As etapas seguintes rodam mesmo assim
- c) A versão vai para produção com alerta

Quiz: teste o que você aprendeu (2/2)

Mais duas questões — depois vamos ao gabarito comentado.

Questão 4

Qual prática NÃO faz parte de um bom CI/CD?

- a) Branches de vida curta
- b) Guardar senhas no repositório
- c) Build reproduzível por um comando

Questão 5

Qual métrica mede o tempo para se recuperar de uma falha?

- a) Frequência de deploy
- b) Tempo até produção
- c) Tempo de recuperação

Como responder

Anote suas respostas antes de virar o slide. Vale mais errar aqui do que em produção: o objetivo é descobrir o que ainda precisa ficar mais claro.

Gabarito comentado

Questão 1 — alternativa b

CI é integrar e validar o código com frequência. Publicar em produção já é papel do CD.

Questão 2 — alternativa b

Na Entrega Contínua a versão fica pronta e alguém decide publicar; na Implantação, o pipeline publica sozinho.

Questão 3 — alternativa a

A falha interrompe o pipeline e notifica a equipe: nada quebrado avança para a etapa seguinte.

Questão 4 — alternativa b

Segredos nunca ficam no repositório. Use os cofres de segredos da ferramenta de CI/CD.

Questão 5 — alternativa c

Tempo de recuperação mede quanto tempo a equipe leva para restabelecer o serviço após uma falha.

Errou alguma?

Tudo bem: cada um desses pontos volta com prática nos laboratórios da disciplina.